

Stack-Entscheidung Raumbuchung

LF 10 Benutzerschnittstellen · Vorbereitung auf das Kundengespräch · Luca Textor, Emilio Schmech
Projekt: Web-Anwendung zur Buchung von Meetingräumen, ca. 300 Mitarbeitende, gemischte IT-Kenntnisse

1 Bewertete Stacks

Kriterium	A Angular 21 + Optimus UI Express.js + MariaDB	B React / Next.js Node + PostgreSQL	C Spring Boot + Thymeleaf MariaDB
Know-how im Team	Angular, Express und die UI-Bibliothek sind bereits im Einsatz – keine Einarbeitung	Neu für uns: JSX, Rendering-Modelle, eigene Projektstruktur	Zweite Sprache (Java) plus Template-Engine, komplett neu
Barrierefreiheit	Optimus UI zielt auf WCAG 2.1 AA – Tastaturbedienung und Kontraste ab Werk	Hängt an der gewählten Komponentenbibliothek, selbst sicherzustellen	Reines HTML, gut machbar, aber jede Komponente von Hand
Lizenz und Kosten	Durchgängig Open Source (MIT), keine Lizenzkosten	Frameworks frei, brauchbare UI-Bibliotheken oft kostenpflichtig	Frei, aber höherer Betriebsaufwand (JVM)
Formulare, Tabellen, Datum/Uhrzeit	DatePicker, Table, MultiSelect als fertige Komponenten enthalten	Muss aus Einzelbibliotheken zusammengestellt werden	Kalender- und Verfügbarkeitsansicht aufwendig nachzubauen
Aufwand bis zum lauffähigen Prototyp	Gering	Mittel	Hoch

2 Entscheidung und Begründung

Wir realisieren mit **Stack A: Angular 21 mit Optimus UI im Frontend, Express.js als REST-Backend, MariaDB als Datenbank** (Benutzer- und Anmeldedaten, Räume, Buchungen mit Datum und Uhrzeit). Anmeldung über eigene Benutzerverwaltung mit bcrypt-Passworthashes und Session-Cookies (*express-session*), Datenbankzugriff mit SQL-Statements (Prepared Statements).

- **Kein Lizenzrisiko.** Optimus UI ist ein MIT-lizenzierter Community-Fork von PrimeNG, das auf ein kommerzielles Closed-Source-Modell umgestellt hat. Der Kunde zahlt keine Lizenzgebühren und bleibt unabhängig von einem Anbieter.
- **Barrierefreiheit ist eingebaut, nicht nachgerüstet.** Die Komponenten zielen auf WCAG 2.1 AA. Das deckt genau die Anforderungen ab, die sich aus unseren Personas ergeben haben – vollständige Tastaturbedienung für geübte Nutzer, ausreichende Kontraste und Schriftgrößen für seltene Nutzer.
- **Eine Sprache im ganzen Stack.** TypeScript beziehungsweise JavaScript in Frontend und Backend senkt den Einarbeitungsaufwand, vermeidet Brüche an der Schnittstelle und erleichtert die spätere Übergabe an den Kunden.
- **Der Anwendungskern ist Formular- und Tabellenarbeit.** Genau dafür bringt Optimus fertige Komponenten mit (DatePicker, Table, MultiSelect). Bei B und C wäre dieser Teil Eigenbau – mehr Aufwand bei höherem Fehlerisiko.
- **Bekannter Stack, belastbare Schätzung.** Wir haben Angular und Express bereits eingesetzt. Das verkürzt nicht nur die Umsetzung, sondern macht unsere Termin- und Aufwandszusagen gegenüber dem Kunden verlässlicher.

3 Risiken, die wir offenlegen

- Optimus UI setzt **Angular 21 oder neuer** voraus – die Anwendung muss auf aktuellem Stand gehalten werden.
- Optimus ist ein **junger Community-Fork**. Fällt die Pflege aus, bleiben der Unstyled-Modus und einzelne Eigenkomponenten als Rückfallweg; ein Wechsel der UI-Bibliothek wäre aber Aufwand.
- Eine **eigene Benutzerverwaltung** bedeutet, dass Passwortsicherheit, Sperrung und Zurücksetzen in unserer Verantwortung liegen. Die Anbindung an das bestehende Firmenkonto wäre die sicherere, aber aufwendigere Alternative – das ist im Gespräch zu klären.

4 Vorbereitung des Kundengesprächs

Ziel des Termins: Bestätigung der Anforderungen, Freigabe der Technologiewahl und Klärung von Anmeldung und Kalenderanbindung. **Argumentation:** keine Lizenzkosten, Barrierefreiheit für alle Mitarbeitenden, Daten bleiben im Haus, eine Technologie bedeutet schnellere Umsetzung und einfachere Wartung.

Offene Fragen an den Kunden:

- Anmeldung über eine eigene Benutzerverwaltung oder über das bestehende Firmenkonto (Active Directory / Microsoft 365)?
- Soll die Buchung in den vorhandenen Kalender (Outlook) übernommen werden?
- Wie viele Räume, welche Kapazitäten und Ausstattung sind zu verwalten?
- Werden wiederkehrende Termine benötigt (z. B. wöchentliches Jour fixe)?
- Wer darf fremde Buchungen ändern oder stornieren – gibt es eine Vertretungsregelung?
- Betrieb auf einem Server im Haus oder gehostet? Wer übernimmt Wartung und Datensicherung?
- Welcher Zeitrahmen und welches Budget stehen zur Verfügung?